

ME 263 — Applied Finite Element Analysis

Course Policy & Information · Aug, 2026

Credits: 2:1 (2 lecture hours + 1 practical).

Meets: Mon / Wed / Fri, 3-4 PM, MMCR, Department of Mechanical Engineering.

Instructor: Ramsharan Rangarajan, rram@iisc.ac.in

Class announcements via WhatsApp — you'll be added using the phone number on the intake form.

Do this now — before you leave today

Complete the ME 263 intake form. Submitting it before the end of this class is **mandatory**. Fill it here: forms.cloud.microsoft/r/xcF7KJ4eN

No prior Python or FEniCSx experience is needed to enrol — see “Expected background” below.



or scan to open the form

About this course

ME 263 is a problem- and code-driven introduction to applied finite element analysis using FEniCSx (DOLFINx 0.11). It follows on from an introductory FE course: you already know the theory — here you learn to compute with it. Concepts are introduced through problems and live code as needed, rather than through slides or an API tour.

Expected background

You should have:

- completed a graduate-level finite element course (**ME 257** or **CE 204**, or equivalent);
- implemented FE code yourself at least once (for example, a Matlab course project). Using a commercial package does *not* count;
- a few project ideas already in mind — you'll shape these into your midterm and final projects.

No prior Python or FEniCSx experience is required. The course teaches both from the ground up — learning FEniCSx is part of the point. If you're rusty in Python, or have never used it, you belong here just as much as anyone. Enrolment is based on the background you attest to in the intake form.

Course structure

- **Lectures (Mon/Wed/Fri):** concept- and code-driven; no slides.
- **Practice sessions:** an in-class, hands-on session after roughly every 2–3 lectures. I assign an exercise, and you work it in the room. **Bring your laptop.** These sessions are not graded — they are for building fluency. Working in groups is encouraged and preferred over using AI; both are allowed.

Assessment (subject to small changes)

Component	Weight
In-class quizzes	30%
Midterm project	20%

Component	Weight
Final project	30%
Lecture notes	10%
Attendance	10%
Total	100%

Quizzes (30%)

Held in class, on paper — expect at least 6–8 across the term. Your lowest one or two scores are dropped, so an occasional miss carries no penalty. There are no make-up quizzes. If you know in advance that you'll miss more than the dropped quizzes cover, tell me beforehand and we'll work something out; an unannounced absence simply counts among your dropped quizzes.

Projects and viva (Midterm 20%, Final 30%)

You propose the topics, build them in FEniCSx, and present them. Each project includes a viva in which you will be questioned on your work. You are graded on what you submit and how you account for it, not on the tools you used getting there, so be ready to explain and reproduce any part of what you turn in.

Lecture notes (10%)

For each session, one student (on a rotating basis) writes up that session as a Jupyter notebook — theory, code, and explanation — and submits it as a pull request to the shared course GitHub repository. I review and merge it; acceptance of your notebook is your grade for this component. The scribe is the one student permitted to use a laptop during that lecture. Classmates are welcome to comment on pull requests, but peer feedback is ungraded.

Attendance (10%)

Attendance is taken at all class meetings, including practice sessions. Your attendance maps to points as follows:

Attendance	Points
80% and above	10
70% to below 80%	8
60% to below 70%	6
50% to below 60%	5
Below 50%	0

Classroom policies

- **Laptops:** not permitted during lectures, except for the student writing that session's notes. Laptops are required for practice sessions.
- **Phones:** No cell phones in class.
- **Auditing:** This course is not open for auditing.

Use of AI

Treat AI as a teaching assistant — use it freely to explain concepts, debug, and deepen your understanding. All in-class assessments are conducted offline, so no AI (or any device) is available during quizzes. On projects, AI use is permitted, but you are graded on your submission and viva, and you should expect to be questioned on any part of your work. If you cannot explain or reproduce what you submitted, that will show.

Academic integrity

Do your own work, and account honestly for any help you use. Any use of unfair means will, at my discretion, either be escalated through the Institute's academic-integrity process or result in your grade being lowered by one step (for example, A to A-) per instance.

Before the first lecture — environment setup

Have a working environment ready before class begins. There is no teaching assistant for this course, so there is no one to fix a broken install mid-lecture — sort it out in advance. A detailed step-by-step guide will be shared; in brief:

- Install **Miniforge** (preferred over full Anaconda). **Windows users:** first install WSL2 + Ubuntu and do everything inside it — a native Windows install will not work for this course.
- Create the class environment and install DOLFINx 0.11, version-pinned:

```
conda create -n fenicsx
conda activate fenicsx
conda install -c conda-forge fenics-dolfinx=0.11.0 mpich
```

- Register the environment as a Jupyter kernel, and install **VS Code** with the Python and Jupyter extensions (plus Remote-WSL on Windows).
- Install **ParaView** for viewing results, and create a **GitHub** account (your id goes in the intake form).
- Smoke test: in a notebook on the fenicsx kernel, run `import dolfinx; print(dolfinx.__version__)` — it should print 0.11.0.

Course content

The course is taught through prototypical problems, with FEniCSx introduced as the mathematics requires it. What follows is a high-level overview.

This outline is provisional and subject to change — topics, their order, and how far we take each may all be adjusted as the course runs.

Module 1 — Core: Poisson, elasticity, and nonlinear problems

Building the full FEniCSx solver pipeline through a sequence of problems: the Poisson equation, linear elasticity, and nonlinear (hyperelastic) problems.

Module 2 — Mixed methods and the inf-sup condition

Multifield formulations, the inf-sup (LBB) condition in continuous and discrete form, and the compatible finite elements it demands — including H(div)-conforming Raviart–Thomas elements.

Module 3 — Reduced-order modeling

Building fast surrogate models that reproduce full-order solutions at a fraction of the cost: proper orthogonal decomposition (POD), reduced bases and POD modes, and Galerkin projection onto the reduced space.

Module 4 — Error estimation and adaptivity (if time permits)

A posteriori error estimation with adaptive mesh refinement.

Project block

Project work, presentations, and individual vivas.

Primary references

Dokken, *The DOLFINx Tutorial*.

Bleyer, *Computational Mechanics Numerical Tours with FEniCSx* (comet-fenicsx).
